Cognizant – upGrad Training Program uGE_.DOTNET FSD(Python)

Week 8 Document

Name: Adapa Krishna Prasad | **Email:** krishnaprasadadapa912@gmail.com | **Batch Name:** B1_Python | **Faculty:** Kunal

---

Date: 08/04/2026

PROBLEM STATEMENT

A startup wants to build a Food Delivery Platform using microservices.

Users should be able to:

- Register/login
- Browse restaurants
- Place food orders
- Track order status

CustomerService/Models/Customer.cs

```
namespace CustomerService.Models;
public class Customer{
   public int Id {get;set;}
   public required string Name {get;set;}
   public required string Email {get;set;}
}
```

CustomerService/Data/CustomerDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using CustomerService.Models;
namespace CustomerService.Data;
public class CustomerDbContext:DbContext{
   public CustomerDbContext(DbContextOptions<CustomerDbContext> opt):base(opt){}
   public DbSet<Customer> Customers {get;set;}
}
```

CustomerService/Controllers/CustomerController.cs

```
using Microsoft.AspNetCore.Mvc;
using CustomerService.Data;
using CustomerService.Models;
using Microsoft.EntityFrameworkCore;
```

```
using System.Threading.Tasks;
namespace CustomerService.Controllers{
    [ApiController]
    [Route("api/customer")]
    public class CustomerController : ControllerBase{
        private readonly CustomerDbContext _db;
        public CustomerController(CustomerDbContext db){
            _db = db;
        }
        [HttpPost]
        public async Task<IActionResult> Create(Customer c){
            _db.Customers.Add(c);
            await _db.SaveChangesAsync();
            return Ok(c);
        }
        [HttpGet("{id}")]
        public async Task<IActionResult> Get(int id){
            var c = await _db.Customers.FindAsync(id);
            if(c==null)
                return NotFound();
            return Ok(c);
        }
    }
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using CustomerService.Data;
var builder=WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<CustomerDbContext>(x=>x.UseSqlite("Data Source=customer.db"));
builder.Services.AddControllers();
var app=builder.Build();
using(var s=app.Services.CreateScope()){
```

```
var db=s.ServiceProvider.GetRequiredService<CustomerDbContext>();
db.Database.EnsureCreated();
}
app.MapControllers();
app.Run("http://localhost:6001");
```

CustomerService.Tests/CustomerTests.cs

```
using Xunit;
using CustomerService.Controllers;
using CustomerService.Models;
using CustomerService.Data;
using Microsoft.EntityFrameworkCore;
using Microsoft.AspNetCore.Mvc;
using System.Threading.Tasks;
public class CustomerTests{
   private CustomerDbContext GetDb(){
      var opt=new DbContextOptionsBuilder<CustomerDbContext>()
      .UseInMemoryDatabase("cust")
      .Options;
      return new CustomerDbContext(opt);
   }
   [Fact]
   public async Task CreateCustomer(){
      var db=GetDb();
      var controller=new CustomerController(db);
      var c=new Customer{
         Name="Ram",
         Email="ram@gmail.com"
      };
      var result=await controller.Create(c);
      Assert.IsType<OkObjectResult>(result);
   }
```

```
    [Fact]
    public async Task GetCustomer(){
        var db=GetDb();
        db.Customers.Add(new Customer{
            Name="Sam",
            Email="sam@gmail.com"
        });
        db.SaveChanges();
        var controller=new CustomerController(db);
        var result=await controller.Get(1);
        Assert.IsType<OkObjectResult>(result);
    }
    [Fact]
    public async Task CustomerNotFound(){
        var db=GetDb();
        var controller=new CustomerController(db);
        var result=await controller.Get(50);
        Assert.IsType<NotFoundResult>(result);
    }
}
```

MenuService/Models/Menu.cs

```
namespace MenuService.Models{
    public class Menu{
        public int Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public double Price { get; set; }
        public int RestaurantId { get; set; }
    }
}
```

MenuService/Data/MenuDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using MenuService.Models;
```

```
namespace MenuService.Data{
   public class MenuDbContext:DbContext{
      public MenuDbContext(DbContextOptions<MenuDbContext> opt):base(opt){}
      public DbSet<Menu> Menus {get;set;}
   }
}
```

MenuService/Controllers/MenuController.cs

```
using Microsoft.AspNetCore.Mvc;
using MenuService.Data;
using MenuService.Models;
using Microsoft.EntityFrameworkCore;
using System.Threading.Tasks;
using System.Linq;
namespace MenuService.Controllers{
   [ApiController]
   [Route("api/menu")]
   public class MenuController:ControllerBase{
      private readonly MenuDbContext _db;
      public MenuController(MenuDbContext db)
      {
         _db=db;
      }
      [HttpPost]
      public async Task<IActionResult> Create(Menu m){
         if(m.Price<=0)
            return BadRequest();
         _db.Menus.Add(m);
         await _db.SaveChangesAsync();
         return Ok(m);
      }
      [HttpGet("restaurant/{id}")]
      public async Task<IActionResult> GetByRestaurant(int id){
```

```
            var data=await _db.Menus
            .Where(x=>x.RestaurantId==id)
            .ToListAsync();
            return Ok(data);
        }
    }
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using MenuService.Data;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<MenuDbContext>(opt =>
    opt.UseSqlite("Data Source=menu.db"));
builder.Services.AddControllers();
var app = builder.Build();
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<MenuDbContext>();
    db.Database.EnsureCreated();
}
app.MapControllers();
app.Run("http://localhost:6003");
```

MenuService.Tests/MenuTests.cs

```
using Xunit;
using MenuService.Controllers;
using MenuService.Models;
using MenuService.Data;
using Microsoft.EntityFrameworkCore;
using Microsoft.AspNetCore.Mvc;
using System.Threading.Tasks;
namespace MenuService.Tests{
    public class MenuTests{
```

```
private MenuDbContext GetDb(){
   var opt=new DbContextOptionsBuilder<MenuDbContext>()
   .UseInMemoryDatabase("menuTestDb")
   .Options;
   return new MenuDbContext(opt);
}
[Fact]
public async Task AddMenu(){
   var db=GetDb();
   var controller=new MenuController(db);
   var m=new Menu
   {
      Name="Burger",
      Price=100,
      RestaurantId=1
   };
   var result=await controller.Create(m);
   Assert.IsType<OkObjectResult>(result);
}
[Fact]
public async Task GetMenu(){
   var db=GetDb();
   db.Menus.Add(new Menu{
      Name="Pizza",
      Price=200,
      RestaurantId=1
   });
   await db.SaveChangesAsync();
   var controller=new MenuController(db);
   var result=await controller.GetByRestaurant(1);
   Assert.IsType<OkObjectResult>(result);
}
```

```
        [Fact]
        public async Task InvalidPrice(){
            var db=GetDb();
            var controller=new MenuController(db);
            var m=new Menu{
                Name="Test",
                Price=-5,
                RestaurantId=1
            };
            var result=await controller.Create(m);
            Assert.IsType<BadRequestResult>(result);
        }
    }
}
```

RestaurantService/Models/Restaurant.cs

```
namespace RestaurantService.Models{
    public class Restaurant{
        public int Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public string Location { get; set; } = string.Empty;
    }
}
```

RestaurantService/Data/RestaurantDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using RestaurantService.Models;
namespace RestaurantService.Data;
public class RestaurantDbContext : DbContext{
    public RestaurantDbContext(DbContextOptions<RestaurantDbContext> options) : base(options){}
    public DbSet<Restaurant> Restaurants { get; set; }
}
```

RestaurantService/Controllers/RestaurantController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
using RestaurantService.Data;
using RestaurantService.Models;
using Microsoft.EntityFrameworkCore;
using System.Threading.Tasks;
namespace RestaurantService.Controllers{
   [ApiController]
   [Route("api/restaurant")]
   public class RestaurantController : ControllerBase{
      private readonly RestaurantDbContext _db;
      public RestaurantController(RestaurantDbContext db){
         _db=db;
      }
      [HttpPost]
      public async Task<IActionResult> Create(Restaurant r){
         _db.Restaurants.Add(r);
         await _db.SaveChangesAsync();
         return Ok(r);
      }
      [HttpGet("{id}")]
      public async Task<IActionResult> Get(int id){
         var r=await _db.Restaurants.FindAsync(id);
         if(r==null)
            return NotFound();
         return Ok(r);
      }
   }
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using RestaurantService.Data;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<RestaurantDbContext>(opt =>
```

```
    opt.UseSqlite("Data Source=restaurant.db"));
builder.Services.AddControllers();
var app = builder.Build();
using (var scope = app.Services.CreateScope()){
    var db = scope.ServiceProvider.GetRequiredService<RestaurantDbContext>();
    db.Database.EnsureCreated();
}
app.MapControllers();
app.Run("http://localhost:6002");
```

RestaurantTests.cs

```
using Xunit;
using RestaurantService.Controllers;
using RestaurantService.Models;
using RestaurantService.Data;
using Microsoft.EntityFrameworkCore;
using Microsoft.AspNetCore.Mvc;
using System.Threading.Tasks;
public class RestaurantTests{
    private RestaurantDbContext GetDb(){
        var opt=new DbContextOptionsBuilder<RestaurantDbContext>()
        .UseInMemoryDatabase("res")
        .Options;
        return new RestaurantDbContext(opt);
    }
    [Fact]
    public async Task AddRestaurant(){
        var db=GetDb();
        var controller=new RestaurantController(db);
        var r=new Restaurant{
            Name="KFC",
            Location="Hyd"
        };
```

```
        var result=await controller.Create(r);
        Assert.IsType<OkObjectResult>(result);
    }
    [Fact]
    public async Task GetRestaurant()
    {
        var db=GetDb();
        db.Restaurants.Add(new Restaurant{
            Name="PizzaHut",
            Location="Hyd"
        });
        db.SaveChanges();
        var controller=new RestaurantController(db);
       var result=await controller.Get(1);
        Assert.IsType<OkObjectResult>(result);
    }
    [Fact]
    public async Task RestaurantNotFound(){
        var db=GetDb();
        var controller=new RestaurantController(db);
        var result=await controller.Get(90);
        Assert.IsType<NotFoundResult>(result);
    }
}
```

OrderService/Models/Order.cs

```
namespace OrderService.Models;
public class Order{
public int Id{get;set;}
public int CustomerId{get;set;}
public int RestaurantId{get;set;}
public int MenuId{get;set;}
}
```

OrderService/Data/OrderDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using OrderService.Models;
namespace OrderService.Data;
public class OrderDbContext : DbContext{
   public OrderDbContext(DbContextOptions<OrderDbContext> options): base(options) {}
   public DbSet<Order> Orders { get; set; }
}
```

OrderService/Controllers/OrderController.cs

```
using Microsoft.AspNetCore.Mvc;
using OrderService.Data;
using OrderService.Models;
namespace OrderService.Controllers;
[ApiController]
[Route("api/orders")]
public class OrderController:ControllerBase{
private readonly OrderDbContext _context;
private readonly HttpClient _client;
public OrderController(OrderDbContext c,IHttpClientFactory f){
_context=c;
_client=f.CreateClient();
}
[HttpPost]
public async Task<IActionResult> Create(Order o){
var cust=await _client.
GetAsync($"http://localhost:6001/api/customers/{o.CustomerId}");
if(!cust.IsSuccessStatusCode)
return BadRequest("Invalid Customer");
var res=await _client.
GetAsync($"http://localhost:6002/api/restaurants/{o.RestaurantId}");
if(!res.IsSuccessStatusCode)
return BadRequest("Invalid Restaurant");
```

```
var menu=await _client.
GetAsync($"http://localhost:6003/api/menu/restaurant/{o.RestaurantId}");
if(!menu.IsSuccessStatusCode)
return BadRequest("Invalid Menu Item");
_context.Orders.Add(o);
await _context.SaveChangesAsync();
return Ok(o);
}
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using OrderService.Data;
var builder=WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<OrderDbContext>(x=>
x.UseSqlite("Data Source=order.db"));
builder.Services.AddControllers();
builder.Services.AddHttpClient();
var app=builder.Build();
using(var s=app.Services.CreateScope()){
var db=s.ServiceProvider.
GetRequiredService<OrderDbContext>();
db.Database.EnsureCreated();
}
app.MapControllers();
app.Run("http://localhost:6004");
```

OrderTests.cs

```
using Xunit;
using Moq;
using Moq.Protected;
using OrderService.Controllers;
using OrderService.Data;
using OrderService.Models;
```

```
using Microsoft.EntityFrameworkCore;
using Microsoft.AspNetCore.Mvc;
using System.Net;
using System.Net.Http;
using System.Threading;
using System.Threading.Tasks;
public class OrderTests{
    private OrderDbContext GetDb(){
        var options = new DbContextOptionsBuilder<OrderDbContext>()
            .UseInMemoryDatabase(databaseName: "TestDb")
            .Options;
        return new OrderDbContext(options);
    }
    [Fact]
    public async Task CreateOrder_ShouldReturnOk_WhenValidData(){
        var db = GetDb();
        var handler = new Mock<HttpMessageHandler>();
        handler.Protected()
        .Setup<Task<HttpResponseMessage>>(
            "SendAsync",
            ItExpr.IsAny<HttpRequestMessage>(),
            ItExpr.IsAny<CancellationToken>()
        )
        .ReturnsAsync(new HttpResponseMessage{
            StatusCode = HttpStatusCode.OK
        });
        var client = new HttpClient(handler.Object);
        var factory = new FakeFactory(client);
        var controller = new OrderController(db, factory);
        var order = new Order{
            CustomerId = 1,
            RestaurantId = 1,
```

```
            MenuId = 1
        };
        var result = await controller.Create(order);
        Assert.NotNull(result);
        Assert.IsType<OkObjectResult>(result);
    }
    [Fact]
    public async Task CreateOrder_ShouldReturnBadRequest_WhenCustomerInvalid(){
        var db = GetDb();
        var handler = new Mock<HttpMessageHandler>();
        handler.Protected()
        .Setup<Task<HttpResponseMessage>>(
            "SendAsync",
            ItExpr.IsAny<HttpRequestMessage>(),
            ItExpr.IsAny<CancellationToken>()
        )
        .ReturnsAsync(new HttpResponseMessage{
            StatusCode = HttpStatusCode.BadRequest
        });
        var client = new HttpClient(handler.Object);
        var factory = new FakeFactory(client);
        var controller = new OrderController(db, factory);
        var order = new Order{
            CustomerId = 99,
            RestaurantId = 1,
            MenuId = 1
        };
        var result = await controller.Create(order);
        Assert.IsType<BadRequestObjectResult>(result);
    }
}
```

FakeHttpClientFactory.cs

```
using System.Net.Http;

public class FakeFactory : IHttpClientFactory{

   private readonly HttpClient _client;

   public FakeFactory(HttpClient client){

      _client = client;

   }

   public HttpClient CreateClient(string name = null){

      return _client;

   }

}
```

Date: 09/04/2026

1.Create & List Create a directory test1 and go inside it. List contents.

2.Multiple Files Create 3 files: a.txt, b.txt, c.txt

3.Add Content Add text "Linux Practice" into a.txt

4.View Content Display content of a.txt

5.Copy File Copy a.txt to copy_a.txt

6.Rename File Rename b.txt → new_b.txt

7.Delete File Delete c.txt

🗁 ◈ **INTERMEDIATE (Directories + Move)**

8.Move File Create folder data and move a.txt into it

9.Nested Directory Create directory structure: project/logs

10. List Recursively Show all files and folders including subfolders

🔐 ◈ **PERMISSIONS**

11.Check Permissions Show permissions of all files

12.Change Permission Make copy_a.txt executable

13.Remove Write Permission Remove write permission from copy_a.txt

🔄 ◈ **REDIRECTION & PIPES**

14.Overwrite File Write "Hello World" into file.txt

15.Append Data Add "Second Line" to file.txt

16.Use Pipe Show only lines containing "Hello" from file.txt

17.Combine Commands Count number of lines in file.txt

⚙ ◈ PROCESS & SYSTEM

18. Current User Print current logged-in user

19.System Date Display current date

20.Running Processes Show all running processes

21.Simple Script Create script hello.sh that prints: Hello Linux

22.Username Script Print current username using script

23.Date Script Print current date using script

24.Variables Store your name in variable and print it

25.Conditional Check if file file.txt exists

26. Loop Print numbers 1 to 5 using loop

27.Combine Script Create script that prints: Hello <username>, Today is <date>

28.Final Challenge Do this in one go:

- Create 3 files
- Add different content
- Create folder
- Move 1 file into folder
- Display content of another file

```
HP@DESKTOP-U8TK2RR MINGW64 ~
$ mkdir test1

HP@DESKTOP-U8TK2RR MINGW64 ~
$ cd test1

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ ls

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ touch a.txt b.txt c.txt

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ echo "Linux Practice" > a.txt

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ cat a.txt
Linux Practice

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ cp a.txt copy_a.txt

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ mv b.txt new_b.txt
```

```
HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ rm c.txt

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ mkdir data

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ mv a.txt data/

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ mkdir -p project/logs

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ ls -R
.:
copy_a.txt  data/  new_b.txt  project/

./data:
a.txt

./project:
logs/

./project/logs:

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ ls -l
total 1
-rw-r--r-- 1 HP 197121 15 Apr  9 12:55 copy_a.txt
drwxr-xr-x 1 HP 197121  0 Apr  9 12:57 data/
-rw-r--r-- 1 HP 197121  0 Apr  9 12:54 new_b.txt
drwxr-xr-x 1 HP 197121  0 Apr  9 12:57 project/

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ chmod +x copy_a.txt

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ chmod -w copy_a.txt

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ echo "Hello World" > file.txt

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ echo "Second Line" >> file.txt

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ cat file.txt | grep "Hello"
Hello World

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ cat file.txt | wc -l
2
```

```
HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ whoami
HP

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ date
Thu Apr  9 13:00:45 IST 2026

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ ps
      PID    PPID    PGID     WINPID   TTY         UID    STIME COMMAND
     1955    1954    1955      11008  pty0      197609 12:53:58 /usr/bin/bash
     1954       1    1954       3240  ?         197609 12:53:57 /usr/bin/mintty
     2062    1955    2062        904  pty0      197609 13:00:53 /usr/bin/ps

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ nano hello.sh

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ chmod +x hello.sh

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ ./hello.sh
Hello Linux

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ nano user.sh

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ chmod +x user.sh

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ ./user.sh
HP

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ nano date.sh

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ chmod +x date.sh

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ ./date.sh
Thu Apr  9 13:04:03 IST 2026

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ name="Krishna"

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ echo $name
Krishna

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
```

```
$ if [ -f file.txt ]
> then
> echo "File Exists"
> else
> echo "Not Found"
> fi
File Exists

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ for i in 1 2 3 4 5
> do
> echo $i
> done
1
2
3
4
5

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ nano info.sh

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ chmod +x info.sh

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ ./info.sh
Hello HP
Today is Thu Apr  9 13:08:56 IST 2026

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ touch x.txt y.txt z.txt

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ echo "File X" > x.txt

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ echo "File Y" > y.txt

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ echo "File Z" > z.txt

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ mkdir folder

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ mv x.txt folder/

HP@DESKTOP-U8TK2RR MINGW64 ~/test1
$ cat y.txt
File Y
```

Date: 10/04/2026

1.Take a number and print its square and cube

```
num = int(input("Enter a number: "))
square = num * num
cube = num * num * num
print(f"The square of num is {square}, and cube is {cube}")
```

2.Take a string and print: first character last character

```
s = "Hello World"
print(s[0],s[-1])
```

3.Print all numbers between 1–20 that are divisible by 3

```
for i in range(1,21):
   if i % 3==0:
      print(i)
```

4.Take 3 numbers and print the largest

```
num1 = int(input("Enter num1:"))
num2 = int(input("Enter num2:"))
num3 = int(input("Enter num3:"))
if(num1 > num2 & num1 > num3):
   print(num1)
elif(num2 > num3):
   print(num2)
else:
   print(num3)
```

5.Create a function that returns True if number > 100, else False

```
def check(n):
   return n > 100
print(check(int(input())))
```

6.Take input and print: "Welcome Admin" if name is "admin" else "Welcome User"

```
name = input("Enter name: ")
if name == "admin":
   print("Welcome Admin")
else:
   print("Welcome User")
```

7.Create a list and print its length without using len()

```
l = [1, 2, 3, 4, 5]
count = 0
for i in l:
   count += 1
print(count)
```

8.Print multiplication table of a number

```
n = int(input("Enter number: "))
for i in range(1, 11):
   print(n * i)
```

9.Count how many numbers in list are greater than 10

```
l = list(map(int, input().split()))
count = 0
for i in l:
    if i > 10:
        count += 1
print(count)
```

10.Take a word and print it 5 times using loop

```
w = input("Enter word: ")
for i in range(5):
    print(w)
```

11.Take a list and create a new list with only odd numbers

```
l = list(map(int, input().split()))
odd = []
for i in l:
    if i % 2 != 0:
        odd.append(i)
print(odd)
```

12.Check if a number is prime

```
n = int(input())
prime = True
if n < 2:
    prime = False
for i in range(2, n):
    if n % i == 0:
        prime = False
print(prime)
```

13.Find second largest number in a list

```
l = list(map(int, input().split()))
mx = l[0]
sm = l[0]
for i in l:
    if i > mx:
        sm = mx
        mx = i
    elif i > sm and i != mx:
        sm = i
print(sm)
```

14.Take a sentence and count: vowels consonants

```
s = input()
v = 0
c = 0
for i in s:
    if i in "aeiouAEIOU":
        v += 1
    elif i.isalpha():
        c += 1
print(v, c)
```

15.Create dictionary: keys = numbers (1–5) values = cube of numbers

```
d = {}
for i in range(1, 6):
    d[i] = i*i*i
print(d)
```

16.Replace all spaces in string with _

```
s = input()
res = ""
for i in s:
    if i == " ":
        res += "_"
    else:
        res += i
print(res)
```

17.Generate 5 random numbers and store in list

```
import random
l = []
for i in range(5):
    l.append(random.randint(1, 100))
print(l)
```

18.Write a program: count how many times a number appears in list

```
l = list(map(int, input().split()))
x = int(input())
count = 0
for i in l:
    if i == x:
        count += 1
print(count)
```

19.Read a file and print only lines that contain word "Python"

```
with open("test.txt", "r") as f:
    for line in f:
        if "Python" in line:
            print(line.strip())
```

20.Handle exception: user enters string instead of number → print custom error

```
try:
    n = int(input())
    print(n)
except:
    print("Invalid input")
```

21.Check if two strings are anagrams ("listen", "silent")

```
a = input()
b = input()
print(sorted(a) == sorted(b))
```

22.Flatten a list: [1, [2,3], [4,5]] → [1,2,3,4,5]

```
l = [1, [2,3], [4,5]]
```

```
res = []
for i in l:
    if type(i) == list:
        for j in i:
            res.append(j)
    else:
        res.append(i)
print(res)
```

23.Create dictionary from two lists: keys = ["a","b","c"] values = [1,2,3]

```
keys = ["a","b","c"]
values = [1,2,3]
d = {}
for i in range(len(keys)):
    d[keys[i]] = values[i]
print(d)
```

24.Find all duplicate elements in a list

```
l = list(map(int, input().split()))
dup = []
seen = []
for i in l:
    if i not in seen:
        seen.append(i)
    else:
        if i not in dup:
            dup.append(i)
print(dup)
```

25.Sort a list without using sort()

```
l = list(map(int, input().split()))
i = 0
while i < len(l):
    j = i + 1
    while j < len(l):
        if l[i] > l[j]:
            temp = l[i]
            l[i] = l[j]
            l[j] = temp
        j += 1
    i += 1
print(l)
```

26.Create a program: reverse each word in a sentence "hello world" → "olleh dlrow"

```
s = input().split()
res = ""
for i in s:
    temp = ""
    for j in i:
        temp = j + temp
    res += temp + " "
print(res.strip())
```

27.Write a lambda function: check if number is even or odd

```
f = lambda x: "Even" if x % 2 == 0 else "Odd"
print(f(int(input())))
```

28.Read CSV and: print rows where age > 25

```
import csv
with open("data.csv", "r") as f:
    r = csv.reader(f)
    next(r)
    for i in r:
        if int(i[1]) > 25:
            print(i)
```

29.Create nested dictionary and: print only values using loop

```
d = {"a":{"x":1,"y":2},"b":{"z":3}}
for i in d:
    for j in d[i]:
        print(d[i][j])
```

30.Take list and: separate into 2 lists:

- **even numbers**
- **odd numbers**

```
l = list(map(int, input().split()))
even = []
odd = []
for i in l:
    if i % 2 == 0:
        even.append(i)
    else:
        odd.append(i)
print(even)
print(odd)
```

Date: 11/04/2026

1.Create a class Vehicle with:
attributes: brand, speed
method: move() → returns "Vehicle is moving"
Create a subclass Car that:
overrides move() → returns "Car is moving fast"
Create object and call method.

```
class Vehicle:
    def __init__(self, brand, speed):
        self.brand = brand
        self.speed = speed
    def move(self):
        return "Vehicle is moving"
class Car(Vehicle):
    def move(self):
        return "Car is moving fast"
c = Car("BMW", 120)
print(c.move())
```

2.Create a class Employee:
attributes: name, salary
method: display_details()
Create subclass Manager:
add attribute: department
override method to include department

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
    def display(self):
        return self.name, self.salary
class Manager(Employee):
    def __init__(self, name, salary, department):
        super().__init__(name, salary)
        self.department = department
    def display(self):
        return self.name, self.salary, self.department
m = Manager("Krishna", 50000, "IT")
print(m.display())
```

3.Create a class Shape:
method area() → returns 0
Create: Rectangle (length, width) , Circle (radius) , Override area() in both.

```
class Shape:
    def area(self):
        return 0
class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width
    def area(self):
        return self.length * self.width
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius * self.radius
r = Rectangle(5, 4)
c = Circle(3)
print(r.area())
print(c.area())
```

4.Create a class BankAccount:
attributes: account_number, balance
method: deposit(), withdraw()
Add validation: cannot withdraw more than balance

```
class BankAccount:
    def __init__(self, account_number, balance):
        self.account_number = account_number
        self.balance = balance
    def deposit(self, amount):
```

```
        self.balance += amount
        return self.balance
    def withdraw(self, amount):
        if amount > self.balance:
            return "Insufficient Balance"
        self.balance -= amount
        return self.balance
b = BankAccount(12345, 1000)
print(b.deposit(500))
print(b.withdraw(300))
print(b.withdraw(2000))
```

🔷 Encapsulation & Special Methods

5.Create a class Student:
private attribute: __marks
method: set_marks() (validation: 0–100)
method: get_marks()

```
class Student:
    def __init__(self):
        self.__marks = 0
    def set_marks(self, marks):
        if marks >= 0 and marks <= 100:
            self.__marks = marks
        else:
            self.__marks = 0
    def get_marks(self):
        return self.__marks
s = Student()
s.set_marks(85)
print(s.get_marks())
```

6.Create a class Book: attributes: title, author , override __str__() → "Title by Author"

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
    def __str__(self):
        return self.title + " by " + self.author
b = Book("Python Basics", "Krishna")
print(b)
```

7.Create a class CartItem: attributes: name, price, quantity ,override __repr__() to display full info

```
class CartItem:
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
        self.quantity = quantity
    def __repr__(self):
        return "Name: " + self.name + ", Price: " + str(self.price) + ", Quantity: " + str(self.quantity)
c = CartItem("Shirt", 500, 2)
print(c)
```

🔷 Iterators & Generators

8.Create a generator that: yields squares of numbers from 1 to n

```
def gen(n):
    for i in range(1, n+1):
        yield i*i
n = int(input())
for i in gen(n):
    print(i)
```

9.Create a generator for: even numbers up to n

```
def gen(n):
    for i in range(1, n+1):
        if i % 2 == 0:
            yield i
n = int(input())
for i in gen(n):
    print(i)
```

10.Create a custom iterator class: that iterates over a list in reverse order

```
class ReverseIterator:
    def __init__(self, data):
        self.data = data
        self.index = len(data)
    def __iter__(self):
        return self
    def __next__(self):
        self.index -= 1
        if self.index >= 0:
            return self.data[self.index]
        raise StopIteration
l = list(map(int, input().split()))
it = ReverseIterator(l)
for i in it:
    print(i)
```

🔷 map, filter, reduce

11.Given a list of numbers: Use map() to: convert all numbers into their cubes

```
from functools import reduce
nums = list(map(int, input().split()))
print(list(map(lambda x: x**3, nums)))
```

12.Given a list of names: Use filter() to: return only names with length > 5

```
names = input().split()
print(list(filter(lambda x: len(x) > 5, names)))
```

13.Given a list: Use reduce() to: find product of all elements

```
from functools import reduce
nums = list(map(int, input().split()))
print(reduce(lambda x, y: x * y, nums))
```

🔷 Decorators

14.Create a decorator: that prints execution time of a function, Apply it to a function that runs a loop

```
import time
```

```
def decorator(func):
    def wrapper():
        start = time.time()
        func()
        end = time.time()
        print("Execution time:", end - start)
    return wrapper
@decorator
def run_loop():
    for i in range(1000000):
        pass
run_loop()
```

15.Create a decorator: that checks if user is "admin" only then allows function execution ,Otherwise print "Access Denied"

```
def decorator(func):
    def wrapper(user):
        if user == "admin":
            func(user)
        else:
            print("Access Denied")
    return wrapper
@decorator
def show(user):
    print("Welcome Admin")
user = input()
show(user)
```

16.Create a JSON file with: multiple users (name, email) ,Read file and: print only emails ending with "@gmail.com"

```
import json
data = [
    {"name": "A", "email": "a@gmail.com"},
    {"name": "B", "email": "b@yahoo.com"},
    {"name": "C", "email": "c@gmail.com"},
    {"name": "D", "email": "d@hotmail.com"}]
with open("users.json", "w") as f:
    json.dump(data, f)
with open("users.json", "r") as f:
    users = json.load(f)
for i in users:
    if i["email"].endswith("@gmail.com"):
        print(i["email"])
```

17.Write a regex program to: extract all email addresses from a string

```
import re
s = input()
emails = re.findall(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}", s)
print(emails)
```